📘 Unidad 07 — Arrays Unidimensionales (Vectores)

Cátedra: Programación en Computación — UTN FRRQ **Carrera:** Ingeniería Electromecánica — 2° año **Docentes:** Longhi Pablo, Talijancic Iván

> **Requisitos previos:** unidades 05 (bucles) y 06 (funciones). Todos los algoritmos de esta unidad se escriben **dentro de funciones**, con parámetros y valor de retorno.

---

1. 🤔 Qué problema resuelven los arrays

Hasta ahora, cada dato que guardábamos necesitaba su propia variable:

```
let measurement1 = 219.4
let measurement2 = 221.8
let measurement3 = 218.2
```

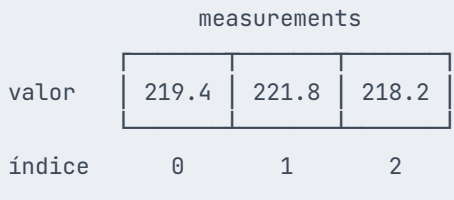
Eso funciona con tres valores. Con 31 mediciones diarias de un mes es inviable, y con una cantidad que el usuario decide en tiempo de ejecución es **imposible**: no podés declarar variables que todavía no sabés cuántas son.

Un **array** (o **vector**, o **arreglo**) es una estructura de datos que guarda una colección de elementos **bajo un mismo nombre**, y accede a cada uno por su **posición**.

```
let measurements = [219.4, 221.8, 218.2]
```

✨ Características

- Los elementos se guardan en **posiciones indexadas**, numeradas desde `0` hasta `n - 1`, donde `n` es la cantidad de elementos.
- Se accede a cada elemento por su **índice**.
- Todos los elementos comparten el nombre de la variable; los distingue el índice.



> ⚠️ **El primer elemento está en el índice 0, no en el 1.** Es la fuente número uno de errores al empezar. Un vector de 3 elementos tiene índices 0, 1 y 2. **No existe** el índice 3.

2. 🏗️ Declaración e inicialización

📦 Declarar un array vacío

```
let numbers = []
```

📩 Declarar con valores iniciales

```
let numbers = [1, 2, 3, 4, 5]
let names = ['Juan', 'Ana', 'Luis']
```

📐 Declarar con una dimensión conocida

Cuando sabés cuántos elementos vas a necesitar pero todavía no qué valores, usás el constructor `Array`:

```
const count = 5
let numbers = new Array(count)   // 5 posiciones, todas vacías
```

Esas posiciones existen pero están **sin inicializar**. Antes de usarlas, cargalas con un bucle:

```
/**
 * Crea un vector de la dimensión indicada, con todos sus elementos en cero.
 * @param {number} size - Cantidad de elementos del vector
 * @returns {number[]} Vector inicializado en cero
 */
const createZeroVector = (size) => {
  const vector = new Array(size)

  for (let i = 0; i < size; i++) {
    vector[i] = 0
  }

  return vector.slice()
}

console.log(createZeroVector(5))   // [ 0, 0, 0, 0, 0 ]
```

> 📌 En esta cátedra los vectores **siempre se inicializan con un bucle**. No usamos `fill()` ni `push()`: el objetivo es que domines el recorrido por índice.

3. 🎯 Acceder y modificar elementos

👀 Leer un elemento

```
let names = ['Juan', 'Ana', 'Luis']

console.log(names[0])   // Juan
console.log(names[2])   // Luis
```

✏️ Modificar un elemento

```
names[1] = 'María'
console.log(names)      // [ 'Juan', 'María', 'Luis' ]
```

📏 Cuántos elementos tiene: `.length`

```
let names = ['Juan', 'Ana', 'Luis']
console.log(names.length)   // 3
```

De acá sale la regla que vas a usar en **todos** los recorridos:

CONCEPTO	VALOR
Cantidad de elementos	`vector.length`
Índice del primer elemento	`0`
Índice del último elemento	`vector.length - 1`

💥 Qué pasa si te vas de rango

```
let names = ['Juan', 'Ana', 'Luis']
console.log(names[3])   // undefined
```

JavaScript **no te avisa con un error**: devuelve `undefined` y el programa sigue. Si después hacés una cuenta con ese valor obtenés `NaN`, y el error aparece mucho más adelante, lejos de donde lo causaste. El control del rango es tu responsabilidad.

---

4. 🔁 Recorrer un array con `for`

Es la forma más común. El `for` es ideal porque sabés exactamente cuántas iteraciones necesitás: tantas como elementos tenga el vector.

```
/**
 * Imprime por consola cada elemento de un vector con su índice.
 * @param {number[]} vector - Vector a printVector
 * @returns {void}
 */
const printVector = (vector) => {
  for (let i = 0; i < vector.length; i++) {
    console.log(`Elemento en índice ${i}: ${vector[i]}`)
  }
}

printVector([10, 20, 30, 40, 50])
```

💻 **Salida:**

```
Elemento en índice 0: 10
Elemento en índice 1: 20
Elemento en índice 2: 30
Elemento en índice 3: 40
Elemento en índice 4: 50
```

⚖️ Por qué la condición es `i < length` y no `i <= length`

EXPRESIÓN	ÚLTIMO `I` QUE EJECUTA	RESULTADO
`i < vector.length`	4	✅ Correcto: recorre 0 a 4
`i <= vector.length`	5	❌ `vector[5]` es `undefined`

Con `.length` igual a 5, los índices válidos son 0, 1, 2, 3 y 4. **Cinco elementos, último índice 4.**

---

5. 🔄 Recorrer un array con `while`

Hace exactamente lo mismo, con el contador manejado a mano:

```
let names = ['Ana', 'Luis', 'María', 'Carlos']
let i = 0

while (i < names.length) {
  console.log(`Nombre en índice ${i}: ${names[i]}`)
  i++
}
```

Tres partes, y si te olvidás de alguna el programa falla:

1. **Inicializar** el contador antes del bucle → `let i = 0`

2. **Condición** de corte → `i < names.length`
3. **Incrementar** el contador dentro del bucle → `i++`

> ⚠️ Si te olvidás el `i++`, la condición nunca se hace falsa y tenés un **bucle infinito**. Cortalo con `Ctrl + C` en la terminal.

🧭 Cuándo conviene cada uno

USÁ `FOR`	USÁ `WHILE`
Recorrer el vector completo	Cortar antes de llegar al final
Sabés la cantidad de iteraciones	La cantidad depende de una condición

---

6. 🧠 Algoritmos fundamentales sobre vectores

Estos cinco patrones resuelven la enorme mayoría de los problemas con vectores. Aprendelos como **patrones**, no de memoria. Todos se escriben como funciones que reciben el vector y devuelven un resultado.

6.1 ➕ Acumular (sumar todos los elementos)

```
/**
 * Calcula la total de todos los elementos de un vector.
 * @param {number[]} vector - Vector de números
 * @returns {number} Suma total de los elementos
 */
const sumAll = (vector) => {
  let total = 0

  for (let i = 0; i < vector.length; i++) {
    total = total + vector[i]      // también: total += vector[i]
  }

  return total
}

const numbers = [5, 10, 15, 20, 25]
console.log(`Suma: ${sumAll(numbers)}`)                        // Suma: 75
console.log(`Promedio: ${sumAll(numbers) / numbers.length}`)    // Promedio: 15
```

🔑 **Clave:** el acumulador arranca en `0` y se declara **fuera** del bucle. Si lo declarás adentro, se reinicia en cada iteración y terminás con el último elemento.

6.2 🔢 Contar según una condición

```
/**
 * Cuenta cuántos elementos pares hay en un vector.
 * @param {number[]} vector - Vector de números enteros
 * @returns {number} Cantidad de elementos pares
 */
const countEven = (vector) => {
  let count = 0

  for (let i = 0; i < vector.length; i++) {
    if (vector[i] % 2 === 0) {
      count++
    }
  }

  return count
}

console.log(countEven([2, 7, 4, 9, 6, 3, 8]))   // 4
```

Igual que acumular, pero el contador sube de a 1 y sólo cuando se cumple la condición.

6.3 🏆 Buscar el máximo y su posición

```
/**
 * Busca el valor máximo de un vector y la posición donde se encuentra.
 * @param {number[]} vector - Vector de números
 * @returns {number[]} Vector de dos elementos: [valor máximo, posición]
 */
const findMax = (vector) => {
  let max = vector[0]      // asumimos que el primero es el máximo
  let position = 0

  for (let i = 1; i < vector.length; i++) {   // arrancamos en 1: el 0 ya lo tomamos
    if (vector[i] > max) {
      max = vector[i]
      position = i
    }
  }

  return [max, position].slice()
}

const result = findMax([15, 42, 7, 81, 23, 56])
console.log(`Máximo: ${result[0]} en la posición ${result[1]}`)
// Máximo: 81 en la posición 3
```

> 📌 **Inicializá el máximo con `vector[0]`, nunca con `0`.** Si inicializás en `0` y todos los valores son negativos, el resultado es `0` — un valor que **no está en el vector**. Es un error clásico y se penaliza como grave. Para el mínimo, la misma lógica con `<`.

Fijate que la función devuelve un **vector de dos elementos** para retornar dos datos. En esta cátedra **no se devuelven objetos ni JSON**: sólo primitivos, vectores y matrices.

6.4 🔍 Búsqueda lineal (¿está este valor?)

```
/**
 * Busca la primera posición en la que aparece un valor dentro de un vector.
 * @param {number[]} vector - Vector donde buscar
 * @param {number} wanted - Valor a buscar
 * @returns {number} Posición del valor, o -1 si no está en el vector
 */
const linearSearch = (vector, wanted) => {
  let position = -1
  let i = 0

  while (i < vector.length && position === -1) {   // corta al encontrarlo
    if (vector[i] === wanted) {
      position = i
    }
    i++
  }

  return position
}

console.log(linearSearch([10, 25, 33, 40], 33))   // 2
console.log(linearSearch([10, 25, 33, 40], 99))   // -1
```

🔑 **Clave:** `position` arranca en `-1` porque es un índice imposible. Así distinguís "no lo encontré" de "está en la posición 0". Y acá el `while` es mejor que el `for`: **corta en cuanto lo encuentra**, sin seguir recorriendo al vacío.

6.5 🎲 Generar un vector aleatorio

Reusamos `rndInt()` de la unidad 06:

```
/**
 * Genera un número entero aleatorio en el rango [min, max].
 * @param {number} min - Valor mínimo del rango (inclusive)
 * @param {number} max - Valor máximo del rango (inclusive)
 * @returns {number} Entero aleatorio entre min y max
 */
const rndInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min

/**
 * Genera un vector de enteros aleatorios dentro de un rango dado.
 * @param {number} size - Cantidad de elementos a generar
 * @param {number} min - Valor mínimo del rango
 * @param {number} max - Valor máximo del rango
 * @returns {number[]} Vector de enteros aleatorios
 */
const getRandomVector = (size, min, max) => {
  const vector = new Array(size)

  for (let i = 0; i < size; i++) {
    vector[i] = rndInt(min, max)
  }

  return vector.slice()
}

console.log(getRandomVector(5, 1, 10))   // ej: [ 3, 7, 2, 9, 5 ]
```

---

7. 🧬 `.slice()`: por qué las funciones devuelven una copia

Acá hay algo que **no** pasa con los números y que tenés que entender bien.

Cuando pasás un **número** a una función, la función recibe una **copia** del valor: modificarla adentro no afecta al original. Cuando pasás un **vector**, la función recibe una **referencia**: apunta al mismo vector en memoria. Si lo modificás adentro, **el original cambia también**.

😱 El problema

```
const doubleAllBad = (vector) => {
  for (let i = 0; i < vector.length; i++) {
    vector[i] = vector[i] * 2      // ⚠️ modifica el vector original
  }
  return vector
}

const original = [1, 2, 3]
const doubled = doubleAllBad(original)

console.log('Original: ', original)    // [ 2, 4, 6 ]  ← ¡se arruinó!
console.log('Duplicado:', doubled)   // [ 2, 4, 6 ]
```

El vector `original` quedó modificado sin que lo pidieras. En un programa grande este error es dificilísimo de encontrar: la función que "sólo calculaba" te cambió los datos de entrada.

✅ La solución: `.slice()`

`.slice()` devuelve una **copia** del vector. Trabajás sobre la copia y el original queda intacto:

```
/**
 * Duplica cada elemento de un vector, sin modificar el vector original.
 * @param {number[]} vector - Vector de números
 * @returns {number[]} Nuevo vector con cada elemento multiplicado por 2
 */
const doubleAll = (vector) => {
  const result = vector.slice()      // copia del original

  for (let i = 0; i < result.length; i++) {
    result[i] = result[i] * 2
  }

  return result
}

const original = [1, 2, 3]
const doubled = doubleAll(original)

console.log('Original: ', original)    // [ 1, 2, 3 ]  ← intacto ✅
console.log('Duplicado:', doubled)   // [ 2, 4, 6 ]
```

🧭 Cuándo usarlo

1. Cuando una función recibe un vector y **no debe modificarlo**.
2. Cuando una función **retorna** un vector: devolvé `vector.slice()`.
3. Siempre que varias funciones trabajen sobre el mismo vector.

> 💡 **Ejercicio mental:** tomá el ejemplo de `doubleAll()`, borrale el `.slice()`, corré el programa y mirá la diferencia en la salida. Entender *por qué* cambia es más valioso que memorizar la regla.

---

8. ⌨️ Cargar un vector desde la consola

Combina todo lo anterior con `prompt`:

```
import { prompt } from './prompt.js'

/**
 * Solicita al usuario la dimensión y los valores de un vector de enteros.
 * @returns {number[]} Vector cargado con los valores ingresados
 */
const readVector = () => {
  const size = parseInt(prompt('Ingrese la dimensión del vector: '))
  const vector = new Array(size)

  for (let i = 0; i < size; i++) {
    vector[i] = parseInt(prompt(`Ingrese el valor del índice ${i}: `))
  }

  return vector.slice()
}

const myVector = readVector()
console.log('\nEl vector ingresado es:')
console.table(myVector)
```

💻 **Salida:**

```
Ingrese la dimensión del vector: 4
Ingrese el valor del índice 0: 1
Ingrese el valor del índice 1: 2
Ingrese el valor del índice 2: 3
Ingrese el valor del índice 3: 4

El vector ingresado es:
┌─────────┬────────┐
│ (index) │ Values │
├─────────┼────────┤
│ 0       │ 1      │
│ 1       │ 2      │
│ 2       │ 3      │
│ 3       │ 4      │
└─────────┴────────┘
```

> 💡 `console.table()` imprime el vector como tabla y es mucho más legible que `console.log()` cuando querés ver índices y valores juntos. Usalo para verificar tus resultados.

No te olvides del `parseInt()`. `prompt` devuelve siempre un **string**. Sin convertir:

```
let a = prompt('Valor: ')   // el usuario escribe 5
console.log(a + 3)          // '53'  ← concatenó, no sumó
```

---

9. 🐛 Errores comunes

ERROR	SÍNTOMA	SOLUCIÓN
Usar `i <= vector.length`	El último valor es `undefined` o aparece un `NaN`	`i < vector.length`
Creer que el primer índice es 1	Se saltea el primer elemento	Los índices arrancan en `0`
Declarar el acumulador dentro del bucle	El resultado es sólo el último elemento	Declaralo **antes** del bucle
Inicializar el máximo en `0`	Con valores negativos devuelve `0`	Inicializalo en `vector[0]`
Olvidar `i++` en un `while`	El programa se cuelga	Incrementá el contador dentro del bucle
Modificar el vector recibido en una función	Los datos originales se corrompen	Trabajá sobre `vector.slice()`
Olvidar `parseInt()` con `prompt`	Las sumas concatenan (`'53'`)	Convertí siempre lo que leas
Comparar con `==` en vez de `===`	Comparaciones que dan `true` sin sentido	Usá siempre `===`
Devolver un objeto `{ max, pos }`	No está permitido en la cátedra	Devolvé un vector: `[max, pos]`

---

10. 📋 Resumen

QUIERO…	CÓMO
Declarar un vector vacío	`let vector = []`
Declarar con dimensión conocida	`let vector = new Array(n)`
Declarar con valores	`let vector = [1, 2, 3]`
Saber cuántos elementos tiene	`vector.length`
Leer el elemento `i`	`vector[i]`
Modificar el elemento `i`	`vector[i] = valor`
Índice del último elemento	`vector.length - 1`
Recorrerlo completo	`for (let i = 0; i < vector.length; i++)`
Recorrerlo pudiendo cortar antes	`while (i < vector.length && !condicion)`
Sumar todo	Acumulador en `0`, declarado fuera del bucle
Contar los que cumplen X	Contador en `0`, `if` adentro del bucle
Buscar máximo/mínimo	Inicializar en `vector[0]`, recorrer desde `i = 1`
Buscar un valor	`position = -1`, cortar al encontrarlo
Devolver dos datos de una función	Un vector: `return [valor, position].slice()`
Copiar un vector	`vector.slice()`
Imprimirlo prolijo	`console.table(vector)`

⛔ Recordá las reglas de la cátedra

Permitido: `for`, `while`, `do-while`, `if` / `else`, `switch-case`, acceso por índice, `.length`, `.slice()`.

No permitido: `map`, `filter`, `reduce`, `forEach`, `find`, `sort`, `flat`, `indexOf`, `splice`, `fill()`, `push()`, retornar objetos/JSON.

No es para complicarte: es para que domines la lógica del recorrido antes de usar herramientas que la resuelven por vos.

📎 Material de la unidad

- Presentación de clase (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/presentacion.pdf)
- Trabajo Práctico (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/tp.pdf)
- Ejemplos de clase (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/ejemplos)